Exercise:

1. Fraction Class

Create a **Fraction** class with the following attributes:

- **Numerator** and **Denominator**

Methods include:

- Default constructor, parameterized constructor (Ensure fraction validity before initialization).
- Method to **simplify** the fraction.
- Method to **add** two fractions.
- Method to **subtract** two fractions.
- Method to **multiply** two fractions.
- Method to **divide** two fractions.

Implement the main method to **execute all above methods**.

2. Time Class

Create a **Time** class with attributes:

- **Hour, Minute, and Second**

Methods include:

- Default constructor, parameterized constructor (Ensure valid time values before initialization).
- Method to **add** hours to the current time and return the updated result.
- Method to **subtract** hours from the current time and return the updated result.

Implement the main method to **execute all above methods**.

3. Circle Class

Create a **Circle** class to define a circle **C(O, r)** with:

- **Center O(a, b)** and **Radius r**

Methods include:

- **area()** → Calculate the area of the circle.
- **perimeter()** → Calculate the perimeter of the circle.
- **testBelongs(x, y)** → Check if point A(x, y) lies inside the circle **C(O, r)** or not.

Implement the main method to **execute all above methods**.

4. Dice Class

Create a **Dice** class to model a six-sided dice and its rolling behavior.

- Each **Dice** object has an attribute **num**, which stores the most recent roll value (a number between **1 and 6**).
- Implement a **roll()** method to simulate rolling the dice and return the outcome.
- Simulate rolling the dice **n** times (where **n** is input by the user).
- Calculate the frequency of each face (1-6) and compute the probability of occurrence for each face.

Implement the main method to **execute all above methods**.

5. First-Degree Polynomial Class

Create a **FirstDegreePolynomial** class to represent polynomials of the form:
F(x) = ax + b (where **a ≠ 0**).

Methods include:

- A method to evaluate **F(x)** for a given x_0.
- A method to find the **root** of the polynomial (solve **F(x) = 0**).
- A method to **add** two first-degree polynomials and return the result.

Implement the main method to **execute all above methods**.

6. Author and Book Management

A program is required to **manage book publishing by authors**.

Author Class

Attributes:

- **Name, Age, Address**
- **A list of published books (List<Book>)**

Methods:

- Default constructor, parameterized constructor.

Book Class

Attributes:

- **Title, Year of Publication, Genre, Price, Number of Copies Sold**

Methods:

- Default constructor, parameterized constructor, and no-argument constructor.

Additional Requirements

Implement methods in the **Author** class to:

- **Calculate the total revenue** from sold books.
- **Calculate revenue by genre** (sum of sales per genre).
- **Calculate revenue by year** (sum of sales per year).

Implement the main method to **execute all above methods**.

7. UITStudent Class

Create a **UITStudent** class with attributes:

- Basic **student card information** and **citizen identification** details (duplicate information can be omitted).
- **GPA (Grade Point Average)**

Methods include:

- Default constructor, parameterized constructor.
- A method to **input student details**.
- A method to **display student personal information**.
- A method to **calculate the student's current age** based on the system date.
- A method to **calculate the expected graduation date**, based on a parameter "CTDT", which accepts values:
    - **"CQ"** → 4 years
    - **"BCU"** → 3.5 years
    - **"CTTT"** → 4.5 years
    - The assumed **start year** = **Birth Year + 18**, with an enrollment month of **September**.

Create a list of 5 UITStudent objects, input their details, and perform the following:

- Print the **age** and **graduation date** of all students in the list.
- Identify the **student with the highest GPA**.
- Print the **graduation order** of students based on their expected graduation year.

Implement the main method to **execute all above methods**.